For example, it's not a completely crazy design to say that Projects might never have data of their own and therefore might never need to be ActiveRecord objects. Somewhat more plausibly, the date-projection code that you're writing now might eventually wind up in some kind of dedicated calculator object separate from the ActiveRecord layer. In either case, there's no need for the design to get ahead of the tests.

let and Expectations

In addition to examining code for potential refactoring, it's a good idea to look at the tests for duplication. In this case you have a single line of common setup—namely, project = Project.new, which is shared between the two tests you've already written. You can fix this and turn your tests into slightly more idiomatic RSpec:

basics/03/spec/models/project_spec.rb

```
require "rails_helper"

RSpec.describe Project do
  let(:project) { Project.new }
  let(:task) { Task.new }

  it "considers a project with no tasks to be done" do
    expect(project).to be_done
  end

  it "knows that a project with an incomplete task is not done" do
    project.tasks << task
    expect(project).not_to be_done
  end

end
```

This version of the test allows me to show two of my favorite parts of RSpec: the let statement and dynamic matchers.

RSpec's let statement cleans up the creation of test data.

Using let, you can make a variable available within the current describe block without having to place it inside the before block and without having to make it an instance variable. I use let all the time. I like that it separates the definition of each variable, that it encourages concise initializations, and that the word let allows me to pretend I'm writing in Scheme or Elm for a brief moment.

Each let method call takes a symbol argument and a block. The symbol can then be called as if it were a local variable: the first call to the symbol lazily invokes the block and caches the result, and subsequent calls within the same spec return the same result without reinvoking the block. When a new spec starts, the value is cleared and the block is reinvoked.